

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМУВАННЯ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 4

Виконав:

студент групи кн-25-1

Рафієв Б. С.

Перевірила:

старший викладач кафедри АІС

Бельська В.Ю.

Тема: Створення та обробка масивів константної розмірності

Мета: отримати навички в створенні та використанні масивів константної розмірності.

Порядок виконання роботи

Необхідно створити код програми засобами мови C++ для виконання дій з масивами відповідно до індивідуальних завдань 1 та 2. Кожне завдання можна вирішувати окремо або разом. Врахувати можливість варіанту багаторазового використання програмного коду.

Завдання 1. Відповідно до обраного варіанту, розробити програмний код, що відповідає завданню. Протестувати програмний код шляхом використання підготовлених тестових наборів даних. Після вдалого тестування додати у програмний код функціонал, що дозволить ввести елементи масиву з клавіатури або заповнити випадковими числами (обирає користувач).

У заданому масиві знайти різницю між першим та останнім непарними елементами і додати отримане число до кожного третього елемента масиву.

Код програми:

```

#include <iostream>
#include <ctime>
#define N 20
using namespace std;

int main() {
    int arr[N];
    int choice;
    int i;

    cout << "Choice type input array:\n";
    cout << "1 - Input\n";
    cout << "2 - Generate random\n";
    cin >> choice;
    switch (choice)
    {
    default: {
        cout << "Error!" << endl;
        return 0;
    }
    case 1: {
        cout << "Input element's of array: ";
        for (i = 0; i < N; i++) cin >> arr[i];
    }
    case 2: {
        int x[N]{ 0 };

        srand((unsigned)time(NULL));
        for (i = 0; i < N; i++) {
            x[i] = rand() % 45 - 25;
            arr[i] = x[i];
        }
    }
    }

    cout << "\nInput array : ";
    for (i = 0; i < N; i++) cout << arr[i] << " ";

    int first = -1, last = -1;
    for (int i = 0; i < N; i++) {
        if (arr[i] % 2 != 0) {
            first = arr[i];
            break;
        }
    }

    int dif = 0;
    if (first == -1) {
        cout << "\nDidn't find %2 != 0 in array!";
        system("pause");
        return 0;
    }

    for (int i = N; i < 0; i--) {
        if (arr[i] % 2 != 0) {
            last = arr[i];
            break;
        }
    }

    dif = first - last;

    cout << "\nThe difference between the first and last odd numbers: " << dif << endl;

    for (int i = 2; i < N; i += 3) {
        arr[i] += dif;
    }

    cout << "Modify array: ";
    for (int i = 0; i < N; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}

```

Рисунок 1 – Код програми до завдання 1

Блок-схема алгоритму має вигляд:

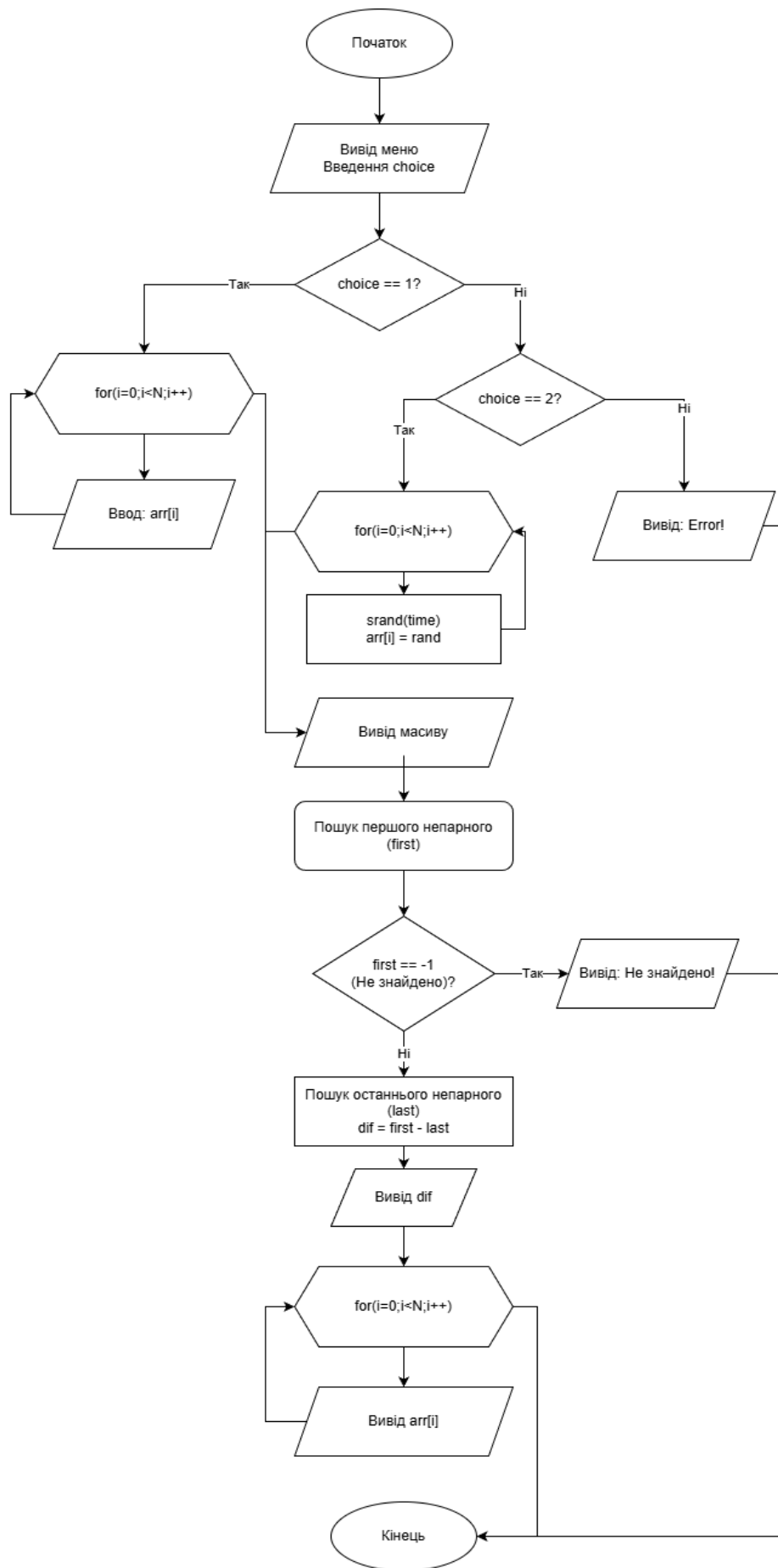


Рисунок 2 – Блок-схема до коду

Тестування програмного коду:

```
Choice type input array:
1 - Input
2 - Generate random
1
Input element's of array:
10
12
-12
0
-24
-11
21
26
9
4
3
7
-4
-8
-1
-23
-44
-11
-8
-13

Input array : 8 14 -16 -5 -25 -13 9 -24 -16 -24 19 -4 -8 2 -23 -11 13 10 12 -4
The difference between the first and last odd numbers: -4
Modify array: 8 14 -20 -5 -25 -17 9 -24 -20 -24 19 -8 -8 2 -27 -11 13 6 12 -4
```

Рисунок 3 – Результат виклику функції 1

```
Choice type input array:
1 - Input
2 - Generate random
2

Input array : -23 -17 -21 -5 9 3 -8 -4 -14 -10 12 -21 10 -2 10 18 -1 -21 17 -24
The difference between the first and last odd numbers: -22
Modify array: -23 -17 -43 -5 9 -19 -8 -4 -36 -10 12 -43 10 -2 -12 18 -1 -43 17 -24

E:\KRN\ATP\2025\Lab\lab4\lab4\x64\Debug\lab4.exe (процесс 27624) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно: _
```

Рисунок 4 – Результат виклику функції 2

Завдання 2. Нехай задано матрицю суміжності графа C розмірності $n \times n$. Необхідно реалізувати крок приведення матриці та провести оцінювання нулів у матриці. Ви вести на екран індекси нуля або нулів матриці вага яких найбільша. Код програми:

```

#include <iostream>
#include <ctime>
#define MAX 20
#define inf 1e12

using namespace std;

void inputMat(double C[MAX][MAX], int n);
void genMat(double C[MAX][MAX], int n);
void outMat(double C[MAX][MAX], int n);
void newMat(double C[MAX][MAX], int n);
bool findWal0(double C[MAX][MAX], int n, int &bestI, int &bestJ, double &bestWeight);

int main() {
    int n, choice;
    int ix[MAX * MAX], jx[MAX * MAX];
    double C[MAX][MAX];

    do {
        cout << "Enter matrix size n < " << MAX << " (n=0 -> Exit): ";
        cin >> n;

        if (n == 0) {
            cout << "Exit!\n";
            break;
        }
        if (n > MAX) {
            cout << "Input n > " << MAX << ", try again\n";
            continue;
        }

        cout << "Choice type input array:\n";
        cout << "1 - Input manually\n";
        cout << "2 - Generate random\n";
        cin >> choice;

        switch (choice) {
            case 1:
                inputMat(C, n);
                break;
            case 2:
                genMat(C, n);
                break;
            default:
                cout << "Incorrect input!\n";
                continue;
        }

        cout << "\nOriginal matrix:\n";
        outMat(C, n);
        newMat(C, n);
        system("pause");

        cout << "\nReduced matrix:\n";
        outMat(C, n);
        system("pause");
        int count = findWal0(C, n, ix, jx);
        cout << "Found zeros: " << count << "\n";
        for (int k = 0; k < count; k++)
            cout << "(" << ix[k] << ", " << jx[k] << ")\n";

        system("pause");
    } while (true);

    return 0;
}

```

Рисунок 5 – Перша частина коду програми до завдання 2

```

void inputMat(double C[MAX][MAX], int n) {
    cout << "Enter adjacency matrix (" << n << "x" << n << "):\n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> C[i][j];
}

void genMat(double C[MAX][MAX], int n) {
    srand((unsigned)time(NULL));
    cout << "Generated adjacency matrix (" << n << "x" << n << "):\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            C[i][j] = rand() % 45 + 1;
        }
    }
}

void outMat(double C[MAX][MAX], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (C[i][j] == inf)
                cout << "inf\t";
            else
                cout << C[i][j] << "\t";
        }
        cout << endl;
    }
}

void newMat(double C[MAX][MAX], int n) {
    for (int i = 0; i < n; i++) {
        C[i][i] = inf;
    }

    for (int i = 0; i < n; i++) {
        double rowMin = inf;
        for (int j = 0; j < n; j++)
            if (C[i][j] < rowMin)
                rowMin = C[i][j];
        for (int j = 0; j < n; j++)
            C[i][j] -= rowMin;
    }

    for (int j = 0; j < n; j++) {
        double colMin = inf;
        for (int i = 0; i < n; i++)
            if (C[i][j] < colMin)
                colMin = C[i][j];
        for (int i = 0; i < n; i++)
            C[i][j] -= colMin;
    }
}

```

Рисунок 6 – Друга частина коду програми до завдання 2

```

int findWal0(double C[MAX][MAX], int n, int bestI[], int bestJ[]) {
    double bestWeight = inf;
    int bestCount = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (C[i][j] == 0) {

                double minRow = inf, minCol = inf;

                for (int k = 0; k < n; k++)
                    if (k != j && C[i][k] < minRow)
                        minRow = C[i][k];

                for (int k = 0; k < n; k++)
                    if (k != i && C[k][j] < minCol)
                        minCol = C[k][j];

                double weight = minRow + minCol;

                if (weight < bestWeight) {
                    bestWeight = weight;
                    bestCount = 0;
                    bestI[bestCount] = i;
                    bestJ[bestCount] = j;
                    bestCount++;
                }
                else if (weight == bestWeight) {
                    bestI[bestCount] = i;
                    bestJ[bestCount] = j;
                    bestCount++;
                }
            }
        }
    }

    return bestCount;
}

```

Рисунок 7 – Третя частина коду програми до завдання 2

Блок-схема алгоритму має вигляд:

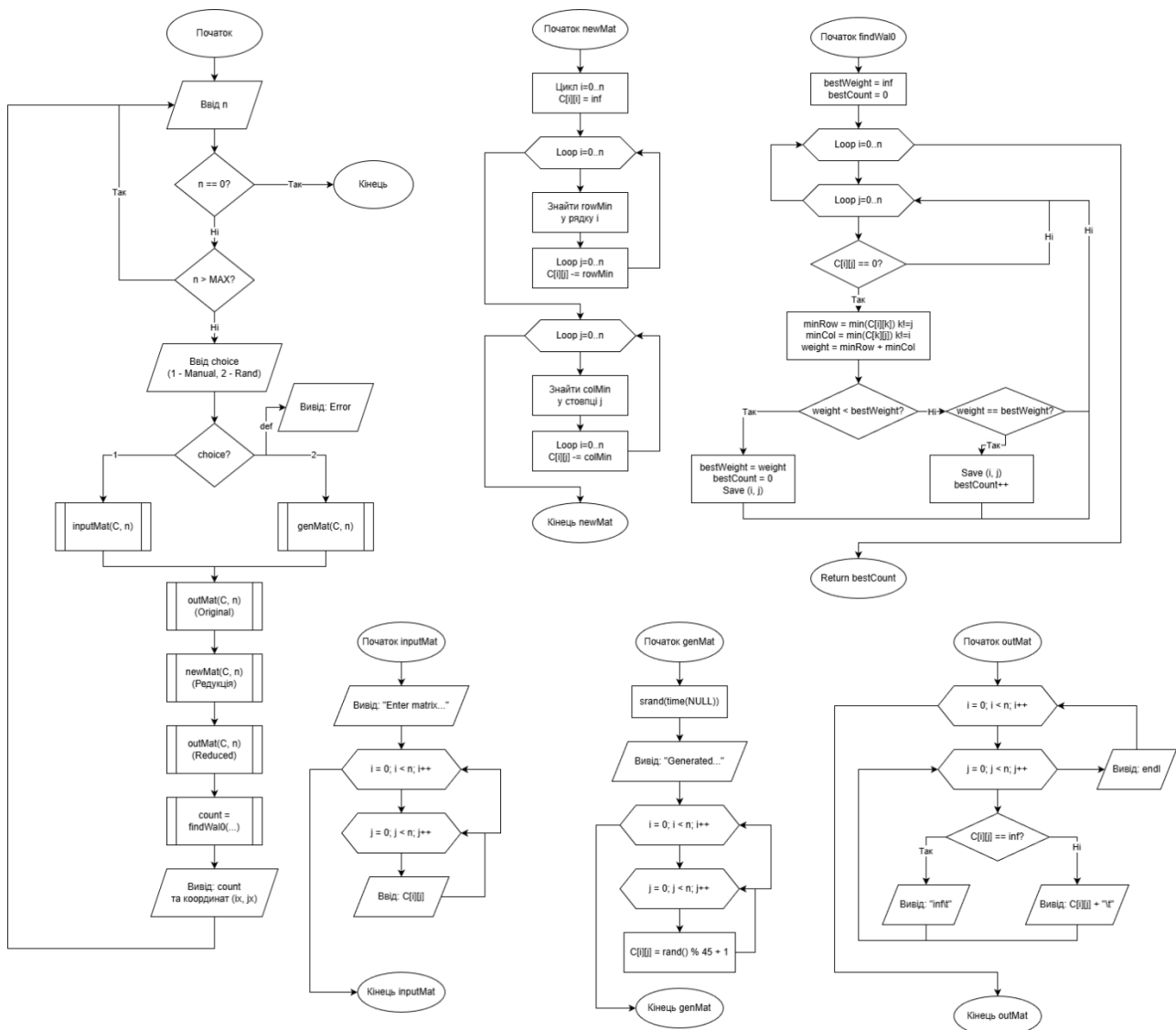


Рисунок 8 – Блок схема до завдання 2

Тестування програмного коду:

```

Enter matrix size n < 20 (n=0 -> Exit): 4
Choice type input array:
1 - Input manually
2 - Generate random
2
Generated adjacency matrix (4x4):

Original matrix:
37      41      36      42
40      37      12      35
27      19      42      15
31      22      23      28
Для продолжения нажмите любую клавишу . . .

Reduced matrix:
1e+12    5      0      6
19      1e+12    0      23
3        4      1e+12    0
0        0      1      1e+12
Для продолжения нажмите любую клавишу . . .

Max weight zero at (3, 0), weight = 3
Для продолжения нажмите любую клавишу . . .

```

Рисунок 9 – Результат виклику функції з генерацією

```

Enter matrix size n < 20 (n=0 -> Exit): 4
Choice type input array:
1 - Input manually
2 - Generate random
1
Enter adjacency matrix (4x4):
1
2
3
4
5
6
7
8
9
0
-1
-2
-3
-4
-5
-6

Original matrix:
1      2      3      4
5      6      7      8
9      0      -1     -2
-3     -4     -5     -6
Для продолжения нажмите любую клавишу . . .

Reduced matrix:
1e+12  0      1      2
0      1e+12  2      3
11     2      1e+12  0
2      1      0      1e+12
Для продолжения нажмите любую клавишу . . .

Max weight zero at (0, 1), weight = 2
Для продолжения нажмите любую клавишу . . .

```

Рисунок 10 – Результат виклику функції з вводом

Висновки: Завдяки цій лабораторній роботі, я отримав навички в створенні та використанні масивів константної розмірності.